

## 3.2.4 Scope of Local and Global Variables

دائرہ کار (Scope) بتاتا ہے کہ متغیر کہاں دکھائی دیتا ہے۔

CS XII

Chapter 3: Programming Fundamentals

Lecture 13 of 17

### 3.2.4 Scope of Local and Global Variables

#### LEARN IT

**Scope** is the region of a program where a variable can be seen and used. Python has two scopes you must know for this chapter.

i) **Local scope.** A **local variable** is created inside a function. It exists only while that function is running. The main program cannot read it.

ii) **Global scope.** A **global variable** is created outside all functions. Any function may **read** it, and the main program can use it too.

#### WORKED EXAMPLE

**Local — the name is invisible outside.**

```
def calc():
    secret = 50
    print(secret)

calc()
print(secret)
```

#### OUTPUT

```
50
Traceback (most recent call last):
  File "local_scope.py", line 5, in <module>
    print(secret)
NameError: name 'secret' is not defined
```

First the function prints 50. Then the main program tries `print(secret)` and fails: `secret` was born inside `calc` and died when `calc` finished.

**Global — readable inside and outside.**

```
x = 10

def show():
    print("Inside function:", x)

show()
print("Outside function:", x)
```

#### OUTPUT

```
Inside function: 10
Outside function: 10
```

**Assignment inside a function creates a new local** with the same name. It does not change the global:

```
x = 10

def show():
    x = 99
    print("Inside:", x)

show()
print("Outside:", x)
```

#### OUTPUT

```
Inside: 99
Outside: 10
```

Inside, `x = 99` is a local. After the call, the global is still 10.

#### REMEMBER IT

**Local = inside the room; global = in the corridor.** Everyone can see the corridor. Things left in a room are gone when you leave. Prefer locals. Too many globals make bugs hard to find because any function might be using them.

#### COMMON MISTAKE

If you only **read** `x` inside the function, Python uses the global. If you **assign** to `x` anywhere in that function, Python treats `x` as local for the whole function. Mixing the two without care causes `UnboundLocalError`. Keep function inputs as parameters and outputs as `return` values instead of editing globals.

#### اردو میں سمجھیں

دائرہ کار (Scope) بتاتا ہے کہ متغیر کہاں دکھائی دیتا ہے۔ لوکل متغیر فنکشن کے اندر بنتا ہے اور فنکشن ختم ہوتے ہی غائب ہو جاتا ہے؛ باہر سے پڑھنے پر `NameError` آتا ہے۔ گلوبل متغیر تمام فنکشنز سے باہر بنتا ہے اور اندر سے پڑھا جا سکتا ہے۔ فنکشن کے اندر اسی نام کو نئی قدر دینے سے نیا لوکل بنتا ہے، گلوبل نہیں بدلتا۔

#### CHECK YOURSELF

1. Where is a local variable defined?
2. In the last example, what does the main program print for `x` after `show()` ?

Answers: 1. Inside a function. 2. 10 (the global is unchanged).

## Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

**MCQ 1.** A local variable lives:

- a. Inside the function
- b. In the whole file always
- c. On the disk
- d. In OSI

**MCQ 2.** A global variable lives:

- a. At module level (the file)
- b. Only in one for-loop
- c. Only in a dict key
- d. Only in RAM of another PC

**MCQ 3.** Reading a global from a function:

- a. Works
- b. Always error
- c. Deletes it
- d. Makes it local

**MCQ 4.** Assigning to a name in a function without global:

- a. Creates a local (does not change the outer)
- b. Always updates global
- c. Syntax error
- d. Opens a file

**MCQ 5.** global x is needed to:

- a. Rebind a global from inside a function
- b. Import math
- c. Draw a pie
- d. Sort a list

**MCQ 6.** Prefer:

- a. Parameters and return, not lots of globals
- b. All globals
- c. No names
- d. eval

## Extra short questions

1. Define local vs global.
2. A classic bug.
3. How to share results instead of globals?
4. Can two functions have locals named total?

## Extra long question

1. Show a 10-line program where a global count is incremented with global, then rewrite it with return so you do not need global. Explain which is clearer.

## Extra answer key

**MCQs:** 1. A 2. A 3. A 4. A 5. A 6. A

### Short

1. Local: created in function, dies when it ends. Global: file-level.
2. `x=1; def f(): x=x+1` without global → `UnboundLocalError`.
3. return values / pass them as arguments.
4. Yes — they are different variables.

### Long

1. Both outputs match. Prefer return. `UnboundLocalError` snippet as a warning.